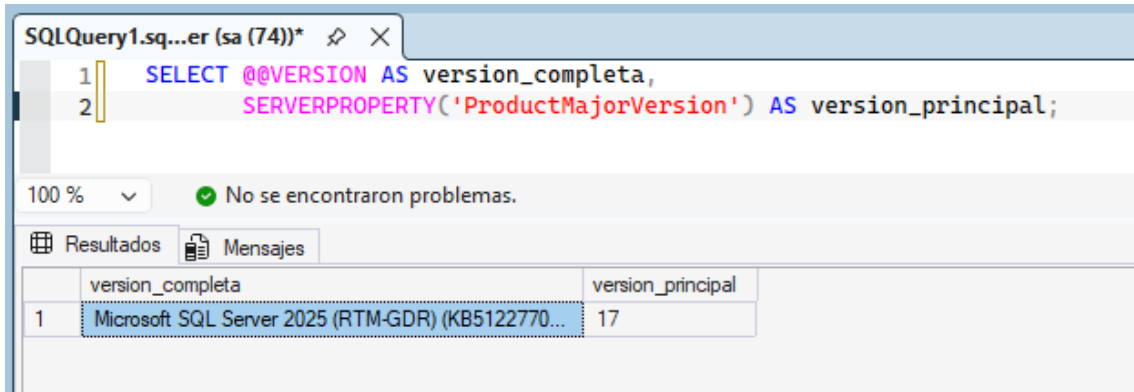


Practica 07

Caso SonoraDB

Comprobar la versión del motor



The screenshot shows a SQL query window with the following code:

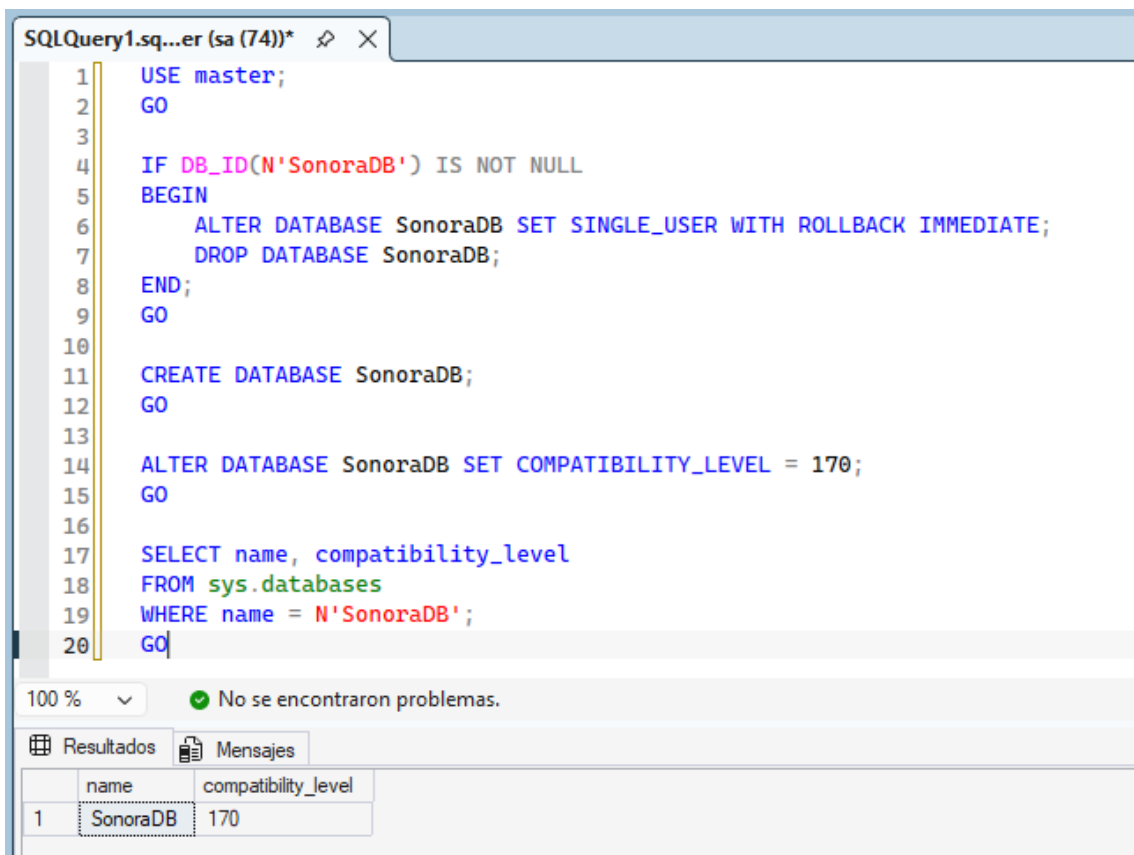
```
1 SELECT @@VERSION AS version_completa,  
2 SERVERPROPERTY('ProductMajorVersion') AS version_principal;
```

Below the query, a status bar indicates "No se encontraron problemas." (No problems found).

The results pane shows the following data:

	version_completa	version_principal
1	Microsoft SQL Server 2025 (RTM-GDR) (KB5122770...	17

Crear BD con nivel de compatibilidad 170



The screenshot shows a SQL script window with the following code:

```
1 USE master;  
2 GO  
3  
4 IF DB_ID(N'SonoraDB') IS NOT NULL  
5 BEGIN  
6     ALTER DATABASE SonoraDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;  
7     DROP DATABASE SonoraDB;  
8 END;  
9 GO  
10  
11 CREATE DATABASE SonoraDB;  
12 GO  
13  
14 ALTER DATABASE SonoraDB SET COMPATIBILITY_LEVEL = 170;  
15 GO  
16  
17 SELECT name, compatibility_level  
18 FROM sys.databases  
19 WHERE name = N'SonoraDB';  
20 GO
```

Below the script, a status bar indicates "No se encontraron problemas." (No problems found).

The results pane shows the following data:

	name	compatibility_level
1	SonoraDB	170

Crear Tablas

```
SQLQuery2.s...DB (sa (74))* X
1  USE SonoraDB;
2  GO
3
4  CREATE TABLE dbo.generos (
5      genero_id      int          NOT NULL,
6      nombre         nvarchar(50) NOT NULL,
7      genero_padre_id int          NULL,
8      CONSTRAINT PK_generos PRIMARY KEY (genero_id),
9      CONSTRAINT FK_generos_genero_padre FOREIGN KEY (genero_padre_id)
10         REFERENCES dbo.generos (genero_id)
11 );
12
13 CREATE TABLE dbo.artistas (
14     artista_id      int          NOT NULL,
15     nombre          nvarchar(100) NOT NULL,
16     pais            char(2)       NOT NULL,
17     genero_id        int          NOT NULL,
18     CONSTRAINT PK_artistas PRIMARY KEY (artista_id),
19     CONSTRAINT FK_artistas_generos FOREIGN KEY (genero_id)
20         REFERENCES dbo.generos (genero_id)
21 );
22
23 CREATE TABLE dbo.canciones (
24     cancion_id      int          NOT NULL,
25     titulo           nvarchar(150) NOT NULL,
26     artista_id       int          NOT NULL,
27     genero_id        int          NOT NULL,
28     duracion_seg     int          NOT NULL,
29     fecha_lanzamiento date        NOT NULL,
30     CONSTRAINT PK_canciones PRIMARY KEY (cancion_id),
31     CONSTRAINT FK_canciones_artistas FOREIGN KEY (artista_id)
32         REFERENCES dbo.artistas (artista_id),
33     CONSTRAINT FK_canciones_generos FOREIGN KEY (genero_id)
34         REFERENCES dbo.generos (genero_id)
35 );
```

```
SQLQuery2.s...DB (sa (74))* X
35 CREATE TABLE dbo.usuarios (
36     usuario_id int NOT NULL,
37     nombre_usuario nvarchar(50) NOT NULL,
38     email nvarchar(150) NOT NULL,
39     pais char(2) NOT NULL,
40     plan_suscripcion nvarchar(20) NOT NULL,
41     fecha_alta date NOT NULL,
42     CONSTRAINT PK_usuarios PRIMARY KEY (usuario_id)
43 );
44 CREATE TABLE dbo.reproducciones (
45     reproduccion_id int NOT NULL,
46     usuario_id int NOT NULL,
47     cancion_id int NULL,
48     fecha_hora datetime2(0) NOT NULL,
49     segundos_escuchados int NOT NULL,
50     dispositivo nvarchar(20) NOT NULL,
51     tipo_contenido nvarchar(10) NOT NULL,
52     CONSTRAINT PK_reproducciones PRIMARY KEY (reproduccion_id),
53     CONSTRAINT FK_reproducciones_usuarios FOREIGN KEY (usuario_id)
54     REFERENCES dbo.usuarios (usuario_id),
55     CONSTRAINT FK_reproducciones_canciones FOREIGN KEY (cancion_id)
56     REFERENCES dbo.canciones (cancion_id),
57     CONSTRAINT CK_reproducciones_tipo_contenido
58     CHECK (tipo_contenido IN (N'Canción', N'Anuncio'))
59 );
60 CREATE TABLE dbo.empleados (
61     empleado_id int NOT NULL,
62     nombre nvarchar(100) NOT NULL,
63     puesto nvarchar(50) NOT NULL,
64     jefe_id int NULL,
65     CONSTRAINT PK_empleados PRIMARY KEY (empleado_id),
66     CONSTRAINT FK_empleados_jefe FOREIGN KEY (jefe_id)
67     REFERENCES dbo.empleados (empleado_id)
68 );
69
```

SQLQuery2.s...DB (sa (74))*

```

69
70  -- Tablas de staging: sin claves ni restricciones
71  CREATE TABLE dbo.stg_reproducciones (
72      reproduccion_id      int,
73      usuario_id           int,
74      cancion_id           int,
75      fecha_hora           datetime2(0),
76      segundos_escuchados  int,
77      dispositivo          nvarchar(20),
78      tipo_contenido       nvarchar(10)
79  );
80
81  CREATE TABLE dbo.stg_usuarios (
82      usuario_id           int,
83      nombre_usuario       nvarchar(50),
84      email                nvarchar(150),
85      pais                 char(2),
86      plan_suscripcion     nvarchar(20),
87      fecha_alta           date
88  );
89  GO
90  SELECT OBJECT_NAME(parent_object_id) AS tabla,
91         name AS restriccion,
92         type_desc AS tipo
93  FROM sys.objects
94  WHERE type IN ('PK', 'F')
95  ORDER BY tabla, tipo DESC, restriccion;

```

100 % No se encontraron problemas.

Resultados Mensajes

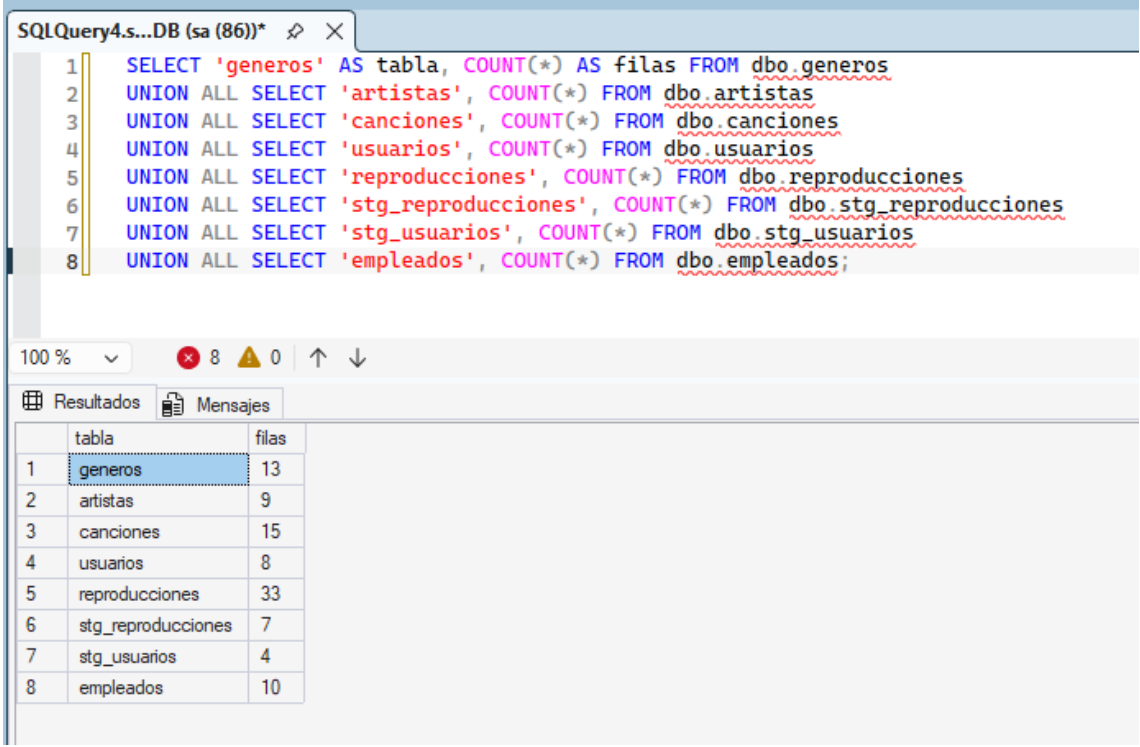
	tabla	restriccion	tipo
1	artistas	PK_artistas	PRIMARY_KEY_CONSTRAINT
2	artistas	FK_artistas_generos	FOREIGN_KEY_CONSTRAINT
3	canciones	PK_canciones	PRIMARY_KEY_CONSTRAINT
4	canciones	FK_canciones_artistas	FOREIGN_KEY_CONSTRAINT
5	canciones	FK_canciones_generos	FOREIGN_KEY_CONSTRAINT
6	empleados	PK_empleados	PRIMARY_KEY_CONSTRAINT
7	empleados	FK_empleados_jefe	FOREIGN_KEY_CONSTRAINT
8	generos	PK_generos	PRIMARY_KEY_CONSTRAINT
9	generos	FK_generos_genero_padre	FOREIGN_KEY_CONSTRAINT
10	reproducciones	PK_reproducciones	PRIMARY_KEY_CONSTRAINT
11	reproducciones	FK_reproducciones_canciones	FOREIGN_KEY_CONSTRAINT
12	reproducciones	FK_reproducciones_usuarios	FOREIGN_KEY_CONSTRAINT
13	usuarios	PK_usuarios	PRIMARY_KEY_CONSTRAINT

Cargar Datos

```
SQLQuery4.s...DB (sa (86)) * X
1  INSERT INTO dbo.generos (genero_id, nombre, genero_padre_id) VALUES
2  (1, N'Música', NULL),
3  (2, N'Pop', 1), (3, N'Urbano', 1), (4, N'Rock', 1), (5, N'Electrónica', 1),
4  (6, N'Reguetón', 3), (7, N'Trap', 3), (8, N'Hip Hop', 3),
5  (9, N'Indie', 4), (10, N'Metal', 4),
6  (11, N'House', 5), (12, N'Techno', 5),
7  (13, N'Dembow', 6);
8
9  INSERT INTO dbo.artistas (artista_id, nombre, pais, genero_id) VALUES
10 (1, N'Luna Roja', 'ES', 2),
11 (2, N'MC Brisa', 'PR', 6),
12 (3, N'Kaiser Beats', 'DE', 12),
13 (4, N'Los Voltios', 'AR', 9),
14 (5, N'Nébula', 'MX', 7),
15 (6, N'Hierro Fundido', 'ES', 10),
16 (7, N'DJ Coral', 'CO', 11),
17 (8, N'Verso Libre', 'ES', 8),
18 (9, N'Sara Cometa', 'ES', 2);
19
20 INSERT INTO dbo.canciones (cancion_id, titulo, artista_id, genero_id, duracion_seg, fecha_lanzamiento) VALUES
21 (101, N'Verano en Bucle', 1, 2, 185, '2026-06-12'),
22 (102, N'Neón', 1, 2, 201, '2025-11-03'),
23 (103, N'Perreo Lunar', 2, 6, 176, '2026-07-01'),
24 (104, N'Calle 23', 2, 6, 190, '2026-03-15'),
25 (105, N'Fábrica 4AM', 3, 12, 412, '2025-09-20'),
26 (106, N'Corriente Alterna', 4, 9, 234, '2026-02-10'),
27 (107, N'Cables Suelos', 4, 9, 198, '2024-05-08'),
28 (108, N'Humo Violeta', 5, 7, 167, '2026-08-22'),
29 (109, N'Diamantes Rotos', 5, 7, 182, '2026-04-30'),
30 (110, N'Forja', 6, 10, 305, '2023-10-31'),
31 (111, N'Marea Alta', 7, 11, 356, '2026-05-19'),
32 (112, N'Rimas de Barrio', 8, 8, 214, '2025-12-01'),
33 (113, N'Galaxia Dembow', 2, 13, 158, '2026-09-04'),
34 (114, N'Sin Cobertura', 5, 7, 175, '2026-09-18'),
35 (115, N'Latido', 7, 11, 330, '2025-07-07');
36
37 INSERT INTO dbo.usuarios (usuario_id, nombre_usuario, email, pais, plan_suscripcion, fecha_alta) VALUES
38 (1, N'alexbeats', N'alex.beats@mail.com', 'ES', N'Premium', '2025-01-10'),
39 (2, N'marta_rock', N'marta.rock@mail.com', 'ES', N'Free', '2025-06-02'),
40 (3, N'juanpi', N'juanpi@mail.com', 'MX', N'Premium', '2024-11-20'),
41 (4, N'sofi.trap', N'sofi.trap@mail.com', 'AR', N'Familiar', '2026-02-14'),
42 (5, N'dani_dj', N'dani.dj@mail.com', 'CO', N'Free', '2026-08-30'),
43 (6, N'lucia.m', N'lucia.m@mail.com', 'ES', N'Premium', '2026-09-01'),
44 (7, N'nachox', N'nachox@mail.com', 'MX', N'Free', '2025-03-18'),
45 (8, N'vale_indie', N'vale.indie@mail.com', 'AR', N'Premium', '2026-05-05');
```

```
SQLQuery4.s...DB (sa (86))* ✕
46 | INSERT INTO dbo.reproducciones (reproduccion_id, usuario_id, cancion_id, fecha_hora, segundos_escuchados, dispositivo, tipo_contenido) VALUES
47 | (1, 1, 103, '2026-09-14 08:15', 176, N'Móvil', N'CanCIÓN'),
48 | (2, 1, 108, '2026-09-14 08:18', 167, N'Móvil', N'CanCIÓN'),
49 | (3, 1, 103, '2026-09-14 09:55', 176, N'Web', N'CanCIÓN'),
50 | (4, 2, 106, '2026-09-14 10:02', 234, N'Web', N'CanCIÓN'),
51 | (5, 2, NULL, '2026-09-14 10:06', 38, N'Web', N'Anuncio'),
52 | (6, 2, 110, '2026-09-14 10:07', 129, N'Web', N'CanCIÓN'),
53 | (7, 3, 105, '2026-09-14 23:10', 412, N'Escritorio', N'CanCIÓN'),
54 | (8, 3, 111, '2026-09-15 00:05', 356, N'Escritorio', N'CanCIÓN'),
55 | (9, 4, 108, '2026-09-15 17:30', 167, N'Móvil', N'CanCIÓN'),
56 | (10, 4, 109, '2026-09-15 17:33', 28, N'Móvil', N'CanCIÓN'),
57 | (11, 4, 103, '2026-09-15 17:35', 176, N'Móvil', N'CanCIÓN'),
58 | (12, 5, NULL, '2026-09-15 21:00', 38, N'Móvil', N'Anuncio'),
59 | (13, 5, 111, '2026-09-15 21:01', 356, N'Móvil', N'CanCIÓN'),
60 | (14, 1, 101, '2026-09-16 07:50', 185, N'Móvil', N'CanCIÓN'),
61 | (15, 6, 101, '2026-09-16 12:00', 185, N'Smart TV', N'CanCIÓN'),
62 | (16, 6, 102, '2026-09-16 12:04', 281, N'Smart TV', N'CanCIÓN'),
63 | (17, 6, 101, '2026-09-16 12:08', 15, N'Smart TV', N'CanCIÓN'),
64 | (18, 7, 112, '2026-09-16 16:20', 214, N'Móvil', N'CanCIÓN'),
65 | (19, 7, NULL, '2026-09-16 16:24', 38, N'Móvil', N'Anuncio'),
66 | (20, 7, 104, '2026-09-16 16:25', 190, N'Móvil', N'CanCIÓN'),
67 | (21, 3, 106, '2026-09-17 09:10', 234, N'Web', N'CanCIÓN'),
68 | (22, 3, 107, '2026-09-17 09:14', 198, N'Web', N'CanCIÓN'),
69 | (23, 2, 107, '2026-09-17 18:45', 198, N'Móvil', N'CanCIÓN'),
70 | (24, 4, 113, '2026-09-18 20:00', 158, N'Móvil', N'CanCIÓN'),
71 | (25, 4, 103, '2026-09-18 20:03', 176, N'Móvil', N'CanCIÓN'),
72 | (26, 1, 113, '2026-09-18 22:30', 158, N'Móvil', N'CanCIÓN'),
73 | (27, 5, 105, '2026-09-19 02:15', 280, N'Móvil', N'CanCIÓN'),
74 | (28, 6, 109, '2026-09-19 11:00', 182, N'Smart TV', N'CanCIÓN'),
75 | (29, 7, 103, '2026-09-19 13:40', 176, N'Móvil', N'CanCIÓN'),
76 | (30, 1, 103, '2026-09-20 09:05', 176, N'Móvil', N'CanCIÓN'),
77 | (31, 3, 112, '2026-09-20 10:30', 214, N'Escritorio', N'CanCIÓN'),
78 | (32, 2, NULL, '2026-09-20 18:00', 38, N'Web', N'Anuncio'),
79 | (33, 2, 106, '2026-09-20 18:01', 234, N'Web', N'CanCIÓN');
80 |
81 | -- Lote con las reproducciones del 21/09, recibido esa noche: las filas 32 y 33 llegan repetidas (ya estaban cargadas)
82 | INSERT INTO dbo.stg_reproducciones (reproduccion_id, usuario_id, cancion_id, fecha_hora, segundos_escuchados, dispositivo, tipo_contenido) VALUES
83 | (32, 2, NULL, '2026-09-20 18:00', 38, N'Web', N'Anuncio'),
84 | (33, 2, 106, '2026-09-20 18:01', 234, N'Web', N'CanCIÓN'),
85 | (34, 6, 113, '2026-09-21 08:00', 158, N'Smart TV', N'CanCIÓN'),
86 | (35, 4, 114, '2026-09-21 18:30', 22, N'Móvil', N'CanCIÓN'),
87 | (36, 3, 110, '2026-09-21 21:00', 385, N'Escritorio', N'CanCIÓN'),
88 | (37, 5, NULL, '2026-09-21 22:00', 38, N'Móvil', N'Anuncio'),
89 | (38, 5, 104, '2026-09-21 22:01', 190, N'Móvil', N'CanCIÓN');
90 |
91 | -- Lote de altas: los usuarios 6 y 8 ya existen
92 | INSERT INTO dbo.stg_usuarios (usuario_id, nombre_usuario, email, pais, plan_suscripcion, fecha_alta) VALUES
93 | (6, N'Lucia.m', N'Lucia.m@mail.com', 'ES', N'Premium', '2026-09-01'),
94 | (8, N'vale.indie', N'vale.indie@mail.com', 'AR', N'Premium', '2026-05-05'),
95 | (9, N'pau.synth', N'pau.synth@mail.com', 'ES', N'Free', '2026-09-21'),
96 | (10, N'kiara.perreo', N'kiara.perreo@mail.com', 'PR', N'Premium', '2026-09-21');
97 |
98 | INSERT INTO dbo.empleados (empleado_id, nombre, puesto, jefe_id) VALUES
99 | (1, N'Irene Salas', N'CEO', NULL),
100 | (2, N'Tomás Vidal', N'CTO', 1),
101 | (3, N'Nuria Campos', N'CMO', 1),
102 | (4, N'Raúl Benet', N'Head of Data', 2),
103 | (5, N'Elena Ruiz', N'Data Engineer', 4),
104 | (6, N'Pablo Ortega', N'Data Engineer', 4),
105 | (7, N'Carmen Lozano', N'Data Analyst', 4),
106 | (8, N'Hugo Marín', N'Backend Lead', 2),
107 | (9, N'Laura Gil', N'Marketing Manager', 3),
108 | (10, N'Andrés Pozo', N'Community Manager', 9);
109 | GO
100 % 51 0
Mensajes
(13 filas afectadas)
(9 filas afectadas)
(15 filas afectadas)
(8 filas afectadas)
(33 filas afectadas)
(7 filas afectadas)
(4 filas afectadas)
(10 filas afectadas)
Hora de finalización: 2026-09-29T09:35:14.8187099+02:00
100 % No se encontraron problemas.
Consulta ejecutada correctamente.
```

Verificar la carga



The screenshot shows a SQL query window titled 'SQLQuery4.s...DB (sa (86))'. The query is a UNION ALL statement that counts the number of rows in eight tables: generos, artistas, canciones, usuarios, reproducciones, stg_reproducciones, stg_usuarios, and empleados. The results are displayed in a table with two columns: 'tabla' and 'filas'.

```
1 SELECT 'generos' AS tabla, COUNT(*) AS filas FROM dbo.generos
2 UNION ALL SELECT 'artistas', COUNT(*) FROM dbo.artistas
3 UNION ALL SELECT 'canciones', COUNT(*) FROM dbo.canciones
4 UNION ALL SELECT 'usuarios', COUNT(*) FROM dbo.usuarios
5 UNION ALL SELECT 'reproducciones', COUNT(*) FROM dbo.reproducciones
6 UNION ALL SELECT 'stg_reproducciones', COUNT(*) FROM dbo.stg_reproducciones
7 UNION ALL SELECT 'stg_usuarios', COUNT(*) FROM dbo.stg_usuarios
8 UNION ALL SELECT 'empleados', COUNT(*) FROM dbo.empleados;
```

	tabla	filas
1	generos	13
2	artistas	9
3	canciones	15
4	usuarios	8
5	reproducciones	33
6	stg_reproducciones	7
7	stg_usuarios	4
8	empleados	10

Ejercicio 1

-- Ejercicio 1

```
SELECT SERVERPROPERTY('ProductMajorVersion') AS versionPrincipal,
compatibility_level FROM sys.databases WHERE name = 'SonoraDB';
GO
```

Ejercicio 2

-- Ejercicio 2

```
SELECT name FROM sys.tables ORDER BY name ASC;
```

Ejercicio 3

-- Ejercicio 3

```
SELECT COLUMN_NAME, DATA_TYPE, CHARACTER_MAXIMUM_LENGTH, IS_NULLABLE FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'reproducciones' ORDER BY ORDINAL_POSITION;
```

Ejercicio 4

-- Ejercicio 4

```
SELECT nombre_usuario, plan_suscripcion, fecha_alta FROM USUARIOS  
WHERE pais = 'ES' ORDER BY fecha_alta ASC
```

Ejercicio 5

-- Ejercicio 5

```
SELECT titulo, duracion_seg, fecha_lanzamiento FROM CANCIONES  
WHERE fecha_lanzamiento >= '2026-01-01' AND fecha_lanzamiento <= '2026-12-31'  
AND duracion_seg >= 170 AND duracion_seg <= 200  
ORDER BY fecha_lanzamiento ASC;
```

Ejercicio 6

-- Ejercicio 6

```
SELECT nombre_usuario FROM USUARIOS WHERE nombre_usuario LIKE '%[_]%' ORDER BY nombre_usuario ASC;  
  
-- El motivo por el que LIKE '%_%' no vale es por que _ es un comodin que representa cualquier caracter
```

Ejercicio 7

-- Ejercicio 7

```
SELECT DISTINCT dispositivo FROM reproducciones ORDER BY dispositivo ASC;
```

Ejercicio 8

-- Ejercicio 8

```
SELECT reproduccion_id, usuario_id, fecha_hora  
FROM reproducciones  
WHERE cancion_id IS NULL;
```

-- cancion_id = NULL no devuelve nada porque en SQL cualquier comparación directa con NULL devuelve desconocido, hay que usar IS NULL

Ejercicio 9

-- Ejercicio 9

```
SELECT TOP (2) WITH TIES reproduccion_id, usuario_id, cancion_id, segundos_escuchados  
FROM reproducciones ORDER BY segundos_escuchados DESC;
```

Ejercicio 10

-- Ejercicio 10

```
SELECT titulo FROM canciones ORDER BY titulo ASC  
OFFSET 5 ROWS FETCH NEXT 5 ROWS ONLY;
```

Ejercicio 11

-- Ejercicio 11

```
SELECT canciones.titulo, canciones.duracion_seg, CASE  
    WHEN duracion_seg < 180 THEN 'corta'  
    WHEN duracion_seg >= 180 AND duracion_seg <= 240 THEN 'Media'  
    WHEN duracion_seg > 240 THEN 'Larga'  
END AS categoria  
FROM canciones  
INNER JOIN artistas ON canciones.artista_id = artistas.artista_id  
WHERE nombre = 'Nébula'  
ORDER BY titulo ASC;
```

Ejercicio 12

-- Ejercicio 12

```
SELECT nombre, CONCAT(nombre, '(', pais, ')') AS etiqueta, LEN(nombre) AS Longitud FROM artistas ORDER BY nombre ASC;
```

Ejercicio 13

-- Ejercicio 13

```
SELECT nombre_usuario, fecha_alta, DATEDIFF(day, fecha_alta, '2026-09-24') AS dias_antiguedad FROM usuarios ORDER BY dias_antiguedad DeSC
```

Ejercicio 14

-- Ejercicio 14

```
SELECT reproduccion_id, CAST(fecha_hora AS date) AS fecha, CAST(fecha_hora AS TIME(0)) AS hora, segundos_escuchados  
from reproducciones WHERE usuario_id = 3;
```

Ejercicio 15

-- Ejercicio 15

```
SELECT COUNT(cancion_id) AS num_canciones, MIN(duracion_seg) AS duracion_min, MAX(duracion_seg) AS duracion_max,  
AVG(duracion_seg) AS media_entera, AVG(duracion_seg * 1.0) AS media_decimal FROM canciones
```

-- Con la columna entera AVG() hace una división entera y quita los decimales así que al multiplicar por 1.0 se fuerza a que tenga decimales

Ejercicio 16

```
-- Ejercicio 16

SELECT COUNT(*) AS total_filas, COUNT(cancion_id) AS con_cancion, COUNT(DISTINCT(cancion_id)) AS canciones_distintas,
COUNT(DISTINCT(usuario_id)) AS usuarios_distintos FROM reproducciones;

-- COUNT(*) cuenta todas las filas de la tabla y COUNT(cancion_id) ignora los nulos (los 4 anuncios).
```

Ejercicio 17

```
-- Ejercicio 17

SELECT dispositivo, COUNT(reproduccion_id) AS reproducciones,
SUM(segundos_escuchados) AS segundos, SUM(segundos_escuchados)/60.0 AS minutos
FROM reproducciones GROUP BY dispositivo ORDER BY reproducciones DESC
```

Ejercicio 18

```
-- Ejercicio 18

SELECT CASE
    WHEN duracion_seg < 180 THEN 'Corta'
    WHEN duracion_seg >= 180 AND duracion_seg <= 240 THEN 'Media'
    WHEN duracion_seg > 240 THEN 'Larga'
END AS Categoria,
COUNT(cancion_id) AS num_canciones, AVG(duracion_seg) AS duracion_media
FROM canciones
GROUP BY CASE
    WHEN duracion_seg < 180 THEN 'Corta'
    WHEN duracion_seg >= 180 AND duracion_seg <= 240 THEN 'Media'
    WHEN duracion_seg > 240 THEN 'Larga'
END
ORDER BY num_canciones DESC, CASE
    WHEN duracion_seg < 180 THEN 'Corta'
    WHEN duracion_seg >= 180 AND duracion_seg <= 240 THEN 'Media'
    WHEN duracion_seg > 240 THEN 'Larga'
END
```

Ejercicio 19

```
-- Ejercicio 19

SELECT usuarios.nombre_usuario, COUNT(tipo_contenido) AS reproducciones, SUM(segundos_escuchados)
FROM reproducciones
INNER JOIN usuarios ON usuarios.usuario_id = reproducciones.usuario_id
WHERE tipo_contenido = 'Canción'
GROUP BY usuarios.nombre_usuario
HAVING COUNT(r.reproduccion_id) > 3 AND SUM(r(segundos_escuchados)) > 800
ORDER BY SUM(segundos_escuchados) DESC;
```

Ejercicio 20

-- Ejercicio 20

```
SELECT usuarios.plan_suscripcion, tipo_contenido, COUNT(reproduccion_id) AS reproduc
FROM reproducciones
INNER JOIN usuarios ON usuarios.usuario_id = reproducciones.usuario_id
GROUP BY usuarios.plan_suscripcion, tipo_contenido
```

Ejercicio 21

-- Ejercicio 21

```
SELECT artistas.nombre, canciones.titulo, generos.nombre
FROM artistas
INNER JOIN canciones ON artistas.artista_id = canciones.artista_id
INNER JOIN generos ON generos.genero_id = canciones.genero_id
WHERE artistas.pais = 'ES'
ORDER BY artistas.nombre, canciones.titulo ASC
```

Ejercicio 22

-- Ejercicio 22

```
SELECT reproducciones.fecha_hora, usuarios.nombre_usuario, canciones.titulo, artistas.nombre
FROM canciones
INNER JOIN reproducciones ON canciones.cancion_id = reproducciones.cancion_id
INNER JOIN usuarios ON reproducciones.usuario_id = usuarios.usuario_id
INNER JOIN artistas ON artistas.artista_id = canciones.artista_id
WHERE reproducciones.fecha_hora >= '2026-09-16' AND reproducciones.fecha_hora < '2026-09-17'
ORDER BY fecha_hora ASC
```

Ejercicio 23

-- Ejercicio 23

```
SELECT reproducciones.fecha_hora, usuarios.nombre_usuario,
COALESCE(canciones.titulo, N'(Anuncio)') AS titulo, COALESCE(artistas.nombre, N'') AS artista
FROM canciones
LEFT JOIN artistas ON artistas.artista_id = canciones.artista_id
RIGHT JOIN reproducciones ON reproducciones.cancion_id = canciones.cancion_id
INNER JOIN usuarios ON usuarios.usuario_id = reproducciones.usuario_id
WHERE reproducciones.fecha_hora >= '2026-09-16' AND reproducciones.fecha_hora < '2026-09-17'
ORDER BY reproducciones.fecha_hora;
```

-- Falta la reproducción del anuncio porque al ser un INNER JOIN requiere que exista cancion_id, y en los anuncios es NULL

Ejercicio 24

-- Ejercicio 24

```
SELECT usuarios.nombre_usuario, usuarios.plan_suscripcion, usuarios.fecha_alta
FROM usuarios
LEFT JOIN reproducciones ON reproducciones.usuario_id = usuarios.usuario_id
WHERE reproducciones.reproduccion_id IS NULL;
```

Ejercicio 25

-- Ejercicio 25

```
SELECT reproducciones.fecha_hora, usuarios.nombre_usuario,
COALESCE(canciones.titulo, '(Anuncio)') AS titulo, COALESCE(artistas.nombre, '-')
FROM canciones
LEFT JOIN reproducciones ON canciones.cancion_id = reproducciones.cancion_id
INNER JOIN usuarios ON reproducciones.usuario_id = usuarios.usuario_id
LEFT JOIN artistas ON artistas.artista_id = canciones.artista_id
WHERE reproducciones.fecha_hora >= '2026-09-16' AND reproducciones.fecha_hora < '2026-09-17'
ORDER BY fecha_hora ASC

-- Al usar COUNT(*), cuenta la fila vacía generada por el LEFT JOIN (devuelve 1); con COUNT(columna) ignora los nulos y devuelve 0.
```

Ejercicio 26

-- Ejercicio 26

```
SELECT empleado.nombre AS empleado,
empleado.puesto,
COALESCE(jefe.nombre, N'(sin jefe)') AS jefe
FROM empleados AS empleado
LEFT JOIN empleados AS jefe ON jefe.empleado_id = empleado.jefe_id
ORDER BY empleado.empleado_id;
```

Ejercicio 27

-- Ejercicio 27

```
SELECT generos.nombre AS genero,
COUNT(*) AS reproducciones_validas,
CAST(SUM(reproducciones.segundos_escuchados) / 60.0 AS decimal(10,1)) AS minutos
FROM reproducciones
INNER JOIN canciones ON canciones.cancion_id = reproducciones.cancion_id
INNER JOIN generos ON generos.genero_id = canciones.genero_id
WHERE reproducciones.segundos_escuchados >= 30
GROUP BY generos.genero_id, generos.nombre
HAVING COUNT(*) > 3
ORDER BY reproducciones_validas DESC;
```

Ejercicio 28

-- Ejercicio 28

```
SELECT pais FROM artistas
UNION
SELECT pais FROM usuarios
ORDER BY pais;
```

```
SELECT pais FROM artistas
UNION ALL
SELECT pais FROM usuarios
ORDER BY pais;
```

-- UNION elimina los registros duplicados para mostrar solo valores únicos, mientras que UNION ALL los mantiene todos.

Ejercicio 29

-- Ejercicio 29

```
CREATE TABLE dbo.playlists (
    playlist_id int IDENTITY(1,1) NOT NULL,
    usuario_id int NOT NULL,
    nombre nvarchar(100) NOT NULL,
    es_publica bit NOT NULL CONSTRAINT DF_playlists_es_publica DEFAULT 0,
    fecha_creacion datetime2(0) NOT NULL CONSTRAINT DF_playlists_fecha_creacion DEFAULT SYSDATETIME(),
    CONSTRAINT PK_playlists PRIMARY KEY (playlist_id),
    CONSTRAINT FK_playlists_usuarios FOREIGN KEY (usuario_id) REFERENCES dbo.usuarios (usuario_id),
    CONSTRAINT UQ_playlists_usuario_nombre UNIQUE (usuario_id, nombre)
);
```

Ejercicio 30

-- Ejercicio 30

```
CREATE TABLE dbo.playlist_canciones (
    playlist_id int NOT NULL,
    cancion_id int NOT NULL,
    fecha_agregada datetime2(0) NOT NULL CONSTRAINT DF_playlist_canciones_fecha_agregada DEFAULT SYSDATETIME(),
    CONSTRAINT PK_playlist_canciones PRIMARY KEY (playlist_id, cancion_id),
    CONSTRAINT FK_playlist_canciones_playlists FOREIGN KEY (playlist_id) REFERENCES dbo.playlists (playlist_id),
    CONSTRAINT FK_playlist_canciones_canciones FOREIGN KEY (cancion_id) REFERENCES dbo.canciones (cancion_id)
);
```

Ejercicio 31

-- Ejercicio 31

```
ALTER TABLE dbo.playlists
ADD descripcion nvarchar(200) NULL;
ALTER TABLE dbo.playlists
ADD CONSTRAINT CK_playlists_nombre_longitud CHECK (LEN(nombre) >= 3);
```

Ejercicio 32

-- Ejercicio 32

```
INSERT INTO dbo.playlists (usuario_id, nombre)
VALUES
(1, N'Perreo Mañanero'),
(2, N'Rock para currar'),
(3, N'Code & Techno');
SELECT playlist_id, usuario_id, nombre, es_publica, descripcion
FROM dbo.playlists;
```

Ejercicio 33

-- Ejercicio 33

```
INSERT INTO dbo.playlist_canciones (playlist_id, cancion_id)
VALUES
(1, 103),
(1, 104),
(1, 113),
(2, 106),
(2, 107),
(2, 110);

INSERT INTO dbo.playlist_canciones (playlist_id, cancion_id)
SELECT 3, canciones.cancion_id
FROM dbo.canciones
INNER JOIN dbo.generos ON generos.genero_id = canciones.genero_id
WHERE generos.nombre IN (N'House', N'Techno');
```

Ejercicio 34

```
-- Ejercicio 34

INSERT INTO dbo.playlist_canciones (playlist_id, cancion_id)
VALUES (1, 103);

-- Msg 2627: Clave primaria duplicada en playlist_canciones.

INSERT INTO dbo.playlist_canciones (playlist_id, cancion_id)
VALUES (1, 999);

-- Msg 547: Clave foránea fallida, la canción 999 no existe en canciones.

INSERT INTO dbo.playlists (usuario_id, nombre)
VALUES (1, N'AB');

-- Msg 547: Restricción CHECK fallida, el nombre debe tener al menos 3 caracteres.

INSERT INTO dbo.playlists (usuario_id, nombre)
VALUES (1, N'Perreo Mañanero');

-- Msg 2627: Restricción UNIQUE fallida, el usuario ya tiene una playlist con ese nombre.
```

Ejercicio 35

```
-- Ejercicio 35

UPDATE dbo.playlists
SET descripcion = N'Guitarras para la oficina'
WHERE playlist_id = 2;

UPDATE dbo.playlists
SET es_publica = 1
FROM dbo.playlists
INNER JOIN dbo.usuarios ON usuarios.usuario_id = playlists.usuario_id
WHERE usuarios.plan_suscripcion = N'Premium';
```

Ejercicio 36

```
-- Ejercicio 36

SELECT playlists.playlist_id,
usuarios.nombre_usuario,
playlists.nombre,
playlists.es_publica,
COUNT(playlist_canciones.cancion_id) AS num_canciones
FROM dbo.playlists
INNER JOIN dbo.usuarios ON usuarios.usuario_id = playlists.usuario_id
LEFT JOIN dbo.playlist_canciones ON playlist_canciones.playlist_id = playlists.playlist_id
GROUP BY playlists.playlist_id, usuarios.nombre_usuario, playlists.nombre, playlists.es_publica
ORDER BY playlists.playlist_id;
```

Ejercicio 37

```
-- Ejercicio 37

DELETE FROM dbo.playlist_canciones
WHERE playlist_id = 2 AND cancion_id = 110;

DELETE FROM dbo.playlists
WHERE playlist_id = 3;

DELETE FROM dbo.playlist_canciones
WHERE playlist_id = 3;

DELETE FROM dbo.playlists
WHERE playlist_id = 3;
```

Ejercicio 38

```
-- Ejercicio 38

BEGIN TRANSACTION;
UPDATE dbo.usuarios
SET plan_suscripcion = N'Premium'
WHERE usuario_id = 2;
SELECT N'Dentro de la transacción' AS momento, plan_suscripcion
FROM dbo.usuarios
WHERE usuario_id = 2;
ROLLBACK TRANSACTION;

SELECT N'Después del ROLLBACK' AS momento, plan_suscripcion
FROM dbo.usuarios
WHERE usuario_id = 2;
```

Ejercicio 39

```
-- Ejercicio 39

DROP TABLE IF EXISTS dbo.playlist_canciones;
DROP TABLE IF EXISTS dbo.playlists;
```